

Configuración Definitiva: Despliegue Remoto de LLM Local

Prof. CERO y Gemini Code Assist

13 de junio de 2026

Índice

1. Introducción	2
2. Fase 1: Conexión Remota desde la Laptop	2
3. Fase 2: Preparación y Compilación en el Servidor Dell R610	2
3.1. Instalación de Dependencias	2
3.2. Clonación y Compilación de llama.cpp	3
4. Fase 3: Descarga del Modelo de Lenguaje (GGUF)	3
5. Fase 4: Ejecución del Servidor de IA Local	3
6. Fase 5: Consumo desde la Laptop (Frontend)	3
7. Fase 6: Acceso Remoto Externo y Seguridad (Opcional)	4
7.1. Flujo de Trabajo Externo	4
7.2. Advertencias Críticas de Seguridad	4
8. Fase 7: Interfaz Gráfica de Chat (Para Usuarios Finales)	5
8.1. Método A: Interfaz Web Integrada (Rápida)	5
8.2. Método B: Frontend Profesional (Experiencia tipo ChatGPT)	5

1. Introducción

Este documento detalla la arquitectura y el procedimiento técnico definitivo para implementar un ecosistema de Inteligencia Artificial local. La arquitectura se basa en un modelo cliente-servidor descentralizado:

- **Frontend (Cliente):** Una Laptop estándar, utilizada exclusivamente para conectarse remotamente, escribir código, y consumir la IA a través de una red local.
- **Backend (Servidor):** Un servidor Dell PowerEdge R610 (sin monitor ni teclado físicos), operando de manera *Headless*, encargado de cargar el modelo en su amplia memoria RAM y ejecutar la inferencia mediante su CPU.

2. Fase 1: Conexión Remota desde la Laptop

Para administrar el servidor Dell R610 sin necesidad de interactuar físicamente con él, utilizaremos el protocolo **SSH (Secure Shell)**.

1. Conecte el servidor Dell R610 y la Laptop a la misma red local (ej. el mismo router Wi-Fi o switch).
2. En la Laptop, abra una terminal (Linux/macOS) o PowerShell/CMD (Windows).
3. Ejecute el siguiente comando para iniciar sesión de forma remota, reemplazando **usuario** y la dirección IP por los correspondientes al servidor Dell:

```
ssh usuario@192.168.1.100
```

Una vez introducida la contraseña, usted tendrá control total sobre la consola del servidor Dell R610.

3. Fase 2: Preparación y Compilación en el Servidor Dell R610

Debido a que los procesadores Intel Xeon de las series 5500/5600 en el servidor R610 no soportan el conjunto de instrucciones **AVX2**, no es posible instalar binarios precompilados de IA. Debemos compilar el motor de inferencia `llama.cpp` desde el código fuente deshabilitando estas instrucciones.

3.1. Instalación de Dependencias

Ejecute los siguientes comandos en la terminal del servidor (a través del SSH):

```
sudo apt update && sudo apt upgrade -y  
sudo apt install -y build-essential cmake git wget curl
```

3.2. Clonación y Compilación de llama.cpp

Descargaremos el código fuente y lo compilaremos asegurando la compatibilidad con nuestra CPU antigua:

```
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
```

```
# Compilacion estricta sin AVX, AVX2 ni FMA
make LLAMA_AVX=0 LLAMA_AVX2=0 LLAMA_FMA=0
```

4. Fase 3: Descarga del Modelo de Lenguaje (GGUF)

Aprovecharemos la inmensa cantidad de memoria RAM (hasta 192 GB) del servidor Dell para compensar la falta de GPU. Descargaremos un modelo cuantizado optimizado para CPU.

Desde la misma terminal SSH (dentro de la carpeta `llama.cpp`), descargue un modelo ligero como Llama-3 (formato GGUF):

```
mkdir models
cd models
wget https://huggingface.co/lmstudio-community/Meta-Llama-3-8B-Instruct-GGUF\
/resolve/main/Meta-Llama-3-8B-Instruct-Q4_K_M.gguf
cd ..
```

5. Fase 4: Ejecución del Servidor de IA Local

Para que la Laptop pueda consumir la Inteligencia Artificial, levantaremos el modelo en el servidor Dell como una API compatible con el estándar de OpenAI.

Ejecute el siguiente comando para iniciar el servidor de inferencia. El parámetro `-host 0.0.0.0` expone el servicio a toda la red local, y el parámetro `-parallel 4` permite que hasta 4 usuarios interactúen simultáneamente sin tener que esperar en fila:

```
./llama-server -m models/Meta-Llama-3-8B-Instruct-Q4_K_M.gguf \
-c 2048 \
--host 0.0.0.0 \
--port 8080 \
--parallel 4
```

Al habilitar el paralelismo, el servidor Dell R610 aprovechará su masiva memoria RAM para mantener los contextos de chat separados para cada usuario, dividiendo la carga de la CPU entre las peticiones activas.

6. Fase 5: Consumo desde la Laptop (Frontend)

El servidor Dell R610 ahora está actuando como el "Cerebro". Puede cerrar la ventana de SSH si ejecutó el proceso en segundo plano (ej. con `tmux` o `screen`).

En su Laptop, ya puede desarrollar aplicaciones en Python (ej. usando LangChain o la librería `openai`) apuntando al servidor Dell:

```

from openai import OpenAI

client = OpenAI(
    base_url="http://192.168.1.100:8080/v1", # IP del servidor Dell
    api_key="no-necesaria"
)

response = client.chat.completions.create(
    model="llama-3",
    messages=[{"role": "user", "content": "Hola, ¿quién eres?"}]
)
print(response.choices[0].message.content)

```

7. Fase 6: Acceso Remoto Externo y Seguridad (Opcional)

Si el servidor Dell R610 se ubica físicamente en una institución (como una Universidad) y cuenta con una IP pública o un sistema de túneles, es posible trabajar con la Laptop desde cualquier lugar (ej. desde casa). En este escenario, el servidor de la universidad actúa como un "servidor en la nube" totalmente privado.

7.1. Flujo de Trabajo Externo

- **Desarrollo en Casa (Frontend):** La Laptop se utiliza como cliente ligero desde el hogar. No requiere altas capacidades de cómputo ni sufre desgaste térmico por procesar modelos pesados.
- **Inferencia en la Universidad (Backend):** Las peticiones (*prompts*) se envían a través de Internet hacia la IP pública o VPN del servidor (`http://IP_PUBLICA:8080/v1`). El servidor utiliza sus recursos masivos para procesar la IA y devuelve la respuesta.

7.2. Advertencias Críticas de Seguridad

Tener un servicio expuesto en la red pública o universitaria conlleva intentos de acceso no autorizados. Es obligatorio implementar las siguientes medidas de protección:

1. **Asegurar la API de IA:** No debe dejar el puerto 8080 abierto sin protección. Al iniciar el servidor de `llama.cpp`, proteja su uso especificando una contraseña mediante el parámetro `-api-key "SU_CLAVE_SECRETA"`. En sus scripts de Python en la Laptop, actualice el cliente de conexión con `api_key="SU_CLAVE_SECRETA"`.
2. **Asegurar el acceso SSH:** Desactive el inicio de sesión por contraseña tradicional en el servicio de administración SSH y exija únicamente **Claves SSH** (*SSH Keys*) para entrar a la consola del servidor.
3. **Uso de Redes Privadas Virtuales (VPN):** Es muy probable que la red universitaria tenga un cortafuegos (*firewall*) institucional estricto que bloquee puertos de red no estándar (como el 8080). Para sortear esto de manera segura, se recomienda instalar una VPN de malla gratuita como **Tailscale** o **ZeroTier** en ambos equipos.

Esto creará un túnel privado y cifrado entre su Laptop en casa y el servidor en la Universidad, garantizando el acceso a la IA sin comprometer la infraestructura de la institución.

8. Fase 7: Interfaz Gráfica de Chat (Para Usuarios Finales)

Para usuarios que no son desarrolladores y simplemente desean interactuar con la IA desde un navegador web (ya sea en una Laptop o en un teléfono celular), existen dos métodos principales que no requieren programar:

8.1. Método A: Interfaz Web Integrada (Rápida)

Por defecto, al ejecutar `llama-server` (como se describe en la Fase 4), el sistema no solo expone una API, sino que también aloja una página web minimalista.

- El usuario solo debe abrir el navegador en su celular o PC y acceder a: `http://192.168.1.100:8080`
- Inmediatamente cargará una interfaz de chat lista para usar.

8.2. Método B: Frontend Profesional (Experiencia tipo ChatGPT)

Para una experiencia de usuario superior (con historial de conversaciones, diseño responsivo para móviles, modo oscuro y gestión de usuarios), se recomienda instalar un *Frontend* de código abierto como **Open WebUI** o **LibreChat**. Estas aplicaciones web actúan como puente: los usuarios acceden a ellas desde sus navegadores, y estas se conectan por detrás a la API del servidor Dell R610 (`http://192.168.1.100:8080/v1`) para el procesamiento del lenguaje.